LEDGERMIND Architecture

Version: 1.3 (Frozen)
Track: Razorpay Buildathon Track 04 — AI Finance Controller
Status: Implementation-ready. No further architectural changes permitted.

Table of Contents

Core Principle

> **Deterministic when provable. AI when reasoning is required. Human when uncertainty remains.**

- **Python** owns all arithmetic, all validation, all linking.
- **AI** classifies discrepancies using structured evidence. It never calculates, never approves, never invents records.
- **Human** is the final authority. Every AI-investigated discrepancy routes to human review.

Financial Formulas

Fee Structure

Method	Formula
UPI	`fee = 0`
Card	`fee = floor(amount * 0.02) + 100`

Table 1 – continued

Method	Formula
Netbanking	`fee = floor(amount * 0.015) + 100`

Rule: Fees are **never** refunded. Not on full refunds. Not on partial refunds.

Tax Derivation

```
tax = floor(fee * 0.18)
```

`tax` is tax **on the fee** (GST on MDR, or TDS). It is **not** transaction-level GST collected from the customer. Transaction-level GST is embedded inside `payment.amount`.

Expected Settlement Amount

```
expected_amount =
    SUM(linked_payment.amount)
  - SUM(linked_refund.amount)
  - SUM(linked_fee)
  - SUM(linked_tax)
```

Difference

```
difference = actual_settlement_amount - expected_amount
```

Rounding Rules

- All amounts stored as **integer paise** (1 rupee = 100 paise).
- All percentage calculations use `floor()`.
- **No floating-point arithmetic anywhere in the system.**

Data Models

CSV Sources

transactions.csv

Column	Type	Required	Notes
`payment_id`	string	Yes	Primary key
`order_id`	string	Yes	Reference only
`amount`	integer (paise)	Yes	Must be > 0
`status`	string	Yes	`captured` or `failed`
`method`	string	Yes	`upi`, `card`, `netbanking`
`fee`	integer (paise)	Yes	Must match deterministic calculation

Table 2 – continued

Column	Type	Required	Notes
`tax`	integer (paise)	Yes	Must match `floor(fee * 0.18)`
`customer_email`	string	No	**PII —never sent to LLM**
`created_at`	ISO 8601	Yes	Normalized to UTC
`settlement_id`	string	No	Cross-check only; not authoritative

settlements.csv

Column	Type	Required	Notes
`settlement_id`	string	Yes	Primary key
`amount`	integer (paise)	Yes	Actual settlement received
`status`	string	Yes	`settled` or `pending`
`utr`	string	Yes	Bank UTR from Razorpay
`created_at`	ISO 8601	Yes	
`settled_at`	ISO 8601	Yes	Must be >= created_at
`linked_payment_ids`	JSON array	Yes	**Authoritative linkage**
`linked_refund_ids`	JSON array	Yes	**Authoritative linkage**

refunds.csv

Column	Type	Required	Notes
`refund_id`	string	Yes	Primary key
`payment_id`	string	Yes	Must exist in transactions
`amount`	integer (paise)	Yes	Must be > 0
`status`	string	Yes	`processed` only
`created_at`	ISO 8601	Yes	

bank_credits.csv

Column	Type	Required	Notes
`utr`	string	No	May be missing in some bank statements
`amount`	integer (paise)	Yes	Must be > 0
`date`	ISO 8601 date	Yes	

Table 5 – continued

Column	Type	Required	Notes
`description`	string	No	**Never sent to LLM**
`bank_account`	string	No	

Internal Models

EvidencePacket (sent to LLM)

```
{
  "evidence_packet_id": "uuid",
  "settlement_id": "SETL_123",
  "expected_amount_paise": 1270000,
  "actual_amount_paise": 1092500,
  "difference_paise": 177500,
  "linked_payments_summary": {
    "count": 2,
    "total_paise": 1500000,
    "methods": ["upi", "card"]
  },
  "linked_refunds_summary": {
    "count": 1,
    "total_paise": 200000
  },
  "fees_summary": {
    "total_paise": 30000,
    "structure_applied": "card: floor(amount*0.02)+100",
    "validation_result": "passed"
  },
  "tax_summary": {
    "total_paise": 5400,
    "derivation_rule": "floor(fee * 0.18)",
    "validation_result": "passed"
  },
  "bank_credit": {
    "utr": "UTR987654",
    "amount_paise": 1092500,
    "date": "2026-08-22"
  },
  "timing": {
    "settlement_created_at": "2026-08-20T10:00:00Z",
    "settled_at": "2026-08-21T08:00:00Z",
    "bank_credited_at": "2026-08-22T14:30:00Z",
    "expected_cycle_days": 2
  },
  "deterministic_checks_passed": ["references_exist", "bank_match", "fee_match", "tax_match"],
  "deterministic_checks_failed": []
}
```

Rules:

- No individual transaction rows.
- No customer emails or PII.
- Amounts are pre-computed by Python. LLM does not calculate.
- `evidence_packet_id` is used to validate LLM citations.

AIResponse (from LLM)

```
{
  "classification": "TIMING_MISMATCH|REFUND_TIMING|UNEXPLAINED",
  "explanation": "string",
  "raw_confidence": 0.0,
  "cited_evidence": ["evidence_packet_id", "timing"],
  "recommended_action": "ESCALATE_TO_HUMAN"
}
```

Critical constraints:

- `extra="forbid"` — LLM cannot inject new fields.
- `validate_assignment=True` — post-construction mutations are validated.
- No financial fields in schema (`expected_amount`, `actual_amount`, `difference`, `fees`, `refunds`, `tax` are all absent).
- `recommended_action` is hardcoded to `ESCALATE_TO_HUMAN`.

Validation Rules

Runs **before** normalization, linking, and reconciliation.

Schema Validation

- All required columns present.
- Correct types (integer for amounts, ISO 8601 for dates).
- `amount` > 0 for all records.
- `status` values from allowed enum.

Bounds Validation

- Dates not in the future.
- `fee` >= 0, `tax` >= 0.
- `refund.amount` <= corresponding `payment.amount` (REFUND_OVERAGE check).

Referential Integrity

- Every `refund.payment_id` exists in `transactions`.
- Every `settlement.linked_payment_ids` element exists in `transactions`.
- Every `settlement.linked_refund_ids` element exists in `refunds`.

Duplicate Detection (within file)

- Duplicate `settlement_id` → `DUPLICATE_SETTLEMENT`
- Duplicate `refund_id` → `DUPLICATE_REFUND`
- Duplicate `payment_id` → `DUPLICATE_PAYMENT`
- Duplicate `utr` in bank_credits → `DUPLICATE_BANK_UTR`

Cross-File Duplicate Detection

- Duplicate `utr` across settlements → `DUPLICATE_UTR`
- Duplicate `utr` across bank_credits → `DUPLICATE_BANK_UTR`

Encoding Resilience

- Strip BOM (`\ufeff`) from file start.
- Strip currency symbols (₹, `Rs.`) and commas from amount strings.
- Accept `DD-MM-YYYY` and `YYYY-MM-DD`; normalize to ISO 8601.

Entity Linking

Authoritative Source of Truth

`settlements.linked_payment_ids` is **authoritative**.

`transactions.settlement_id` is a **cross-check only**.

Linking Resolution Order

1. Build index: `payment_id → transaction`.
2. For each settlement, resolve `linked_payment_ids` against index.
3. For each transaction with `settlement_id` set, verify it appears in that settlement's `linked_payment_ids`.
4. Resolve `linked_refund_ids` against `refund_id → refund` index.
5. **Link bank credit to settlement:**
   - **Primary:** `settlements.utr == bank_credits.utr` + amount match + date within 2 days
   - **Fallback:** amount match + date within 2 days (if UTR missing)
   - **Failure:** `BANK_MISMATCH`

Linkage Errors (Deterministic)

Error	Trigger
`MISSING_REFERENCE`	Linked ID not found in uploaded data
`LINKAGE_MISMATCH`	`transactions.settlement_id` disagrees with `settlements.linked_payment_ids`
`ORPHAN_PAYMENT`	Payment has `settlement_id` pointing to non-existent settlement
`PAYMENT_OVERCLAIM`	Same `payment_id` in multiple settlements' `linked_payment_ids`
`REFUND_OVERAGE`	Sum of refunds for a payment > payment amount

Deterministic Reconciliation Engine

The engine is **authoritative**. It never delegates math to the AI.

Checks (in order)

1. Schema & validation
2. Duplicate detection
3. Reference existence
4. Linkage consistency
5. Fee validation (`fee == floor(amount * rate) + fixed`)
6. Tax validation (`tax == floor(fee * 0.18)`)
7. Bank credit existence
8. UTR cross-check
9. Amount cross-check
10. Expected amount calculation
11. Difference calculation

Outcomes

Outcome	Condition	AI Called?	Human Review?
`CLEAN_MATCH`	difference == 0, all checks pass	No	No
`MISSING_REFERENCE`	Linked ID not found	No	Yes
`DUPLICATE_SETTLEMENT`	Duplicate settlement_id	No	Yes
`DUPLICATE_UTR`	Duplicate UTR across settlements	No	Yes
`DUPLICATE_REFUND`	Duplicate refund_id	No	Yes
`DUPLICATE_PAYMENT`	Duplicate payment_id	No	Yes
`DUPLICATE_BANK_UTR`	Duplicate UTR in bank_credits	No	Yes
`PAYMENT_OVERCLAIM`	Payment in multiple settlements	No	Yes
`ORPHAN_PAYMENT`	Payment points to missing settlement	No	Yes
`LINKAGE_MISMATCH`	Cross-check fails	No	Yes
`REFUND_OVERAGE`	Refunds exceed payment	No	Yes
`FEE_MISMATCH`	Fee doesn’ t match rule	No	Yes
`TAX_INCONSISTENCY`	Tax doesn’ t match `floor(fee * 0.18)`	No	Yes
`BANK_MISMATCH`	No bank credit found	No	Yes
`BANK_UTR_MISSING`	Found by amount+date, UTR missing/ mismatched	No	Yes
`MATH_DISCREPANCY`	All checks pass, difference != 0	**Yes**	Yes

Engine Failure Handling

```
try:
    result = run_deterministic_engine(settlement)
except Exception:
    result = {"decision": "UNPROCESSED", "escalate_to_human": True}
```

Never crash mid-batch. Never leave a settlement in undefined state.

AI Investigator

What the AI Does

- Receives **structured evidence packet** (never raw CSV, never free text, never PII).
- Classifies discrepancy into AI exception type.
- Provides explanation grounded in cited evidence.
- Outputs raw confidence score (0.0–1.0).
- **Never** calculates amounts. **Never** approves. **Never** invents records.

What Python Does

- Builds evidence packet.
- Pre-computes all arithmetic.
- Validates AI evidence citations.
- Computes final confidence tier.
- **Never** auto-approves. All AI outputs → human review queue.

What the Human Does

- Reviews every AI-investigated discrepancy.
- Approves or rejects AI explanations.
- Handles all deterministic exceptions.
- **Only** final financial authority.

AI Exception Types

Type	Trigger
`TIMING_MISMATCH`	`bank_credited_date - settled_at_date > expected_cycle_days`
`REFUND_TIMING`	Refund created within 24h of settlement boundary
`UNEXPLAINED`	No evidence supports any explanation

Confidence Policy

```
def compute_final_confidence(ai_response, evidence_packet):
    # 1. Validate citations
    valid = all(eid in evidence_packet.evidence_ids
                for eid in ai_response.cited_evidence)
    if not valid:
```

```
        return 0.0, "LOW", "HALLUCINATED_EVIDENCE"

    # 2. Accept LLM confidence
    raw = max(0.0, min(1.0, ai_response.raw_confidence))

    # 3. Tier
    if raw >= 0.7: tier = "HIGH"
    elif raw >= 0.4: tier = "MEDIUM"
    else: tier = "LOW"

    return raw, tier, None
```

Routing: ALL AI cases → human review queue. Confidence affects **only queue priority**.

Batch-Level AI Pattern Analysis

After individual settlements are processed, AI analyzes the **entire batch** for cross-settlement patterns.

Constrained Pattern Types

1. `SYSTEMATIC_FEE_ROUNDING` — same fee discrepancy across multiple settlements
2. `REPEATED_BANK_DELAY` — same timing gap pattern across dates
3. `REFUND_CLUSTER` — multiple refund timing issues on same date
4. `REPEATED_UNEXPLAINED_GAP` — multiple unexplained gaps with similar amounts

Input

Aggregated batch summary: counts by exception type, date distribution.

Output

Pattern list with affected settlement IDs, confidence, recommended action.

Safe: Does not approve anything. Surfaces patterns humans miss.

Decision States

State	Produced By	Final?	Human Review?
`CLEAN_MATCH`	Deterministic engine	Yes	No
`DETERMINISTIC_EXCEPTION`	Deterministic engine	Yes	Yes
`REVIEW_REQUIRED`	AI investigator	No	Yes
`UNRESOLVED`	AI investigator or LLM failure	No	Yes
`EXPLAINED`	AI + human approval	Yes	Yes (human clicked approve)
`UNPROCESSED`	Engine crash	No	Yes

Rules:

- `CLEAN_MATCH` only when `difference == 0` and all checks pass.
- AI never produces final financial approval.
- Human click "Approve" moves `REVIEW_REQUIRED` → `EXPLAINED`.

Audit Trail

Type: Simple append-only records in SQLite.

Schema:

```
CREATE TABLE audit_log (
    id TEXT PRIMARY KEY,
    upload_hash TEXT NOT NULL,
    settlement_id TEXT NOT NULL,
    timestamp TEXT NOT NULL,
    decision_state TEXT NOT NULL,
    payload_json TEXT NOT NULL
);
```

- One record per settlement per run.
- Append-only. Never update.
- `upload_hash` groups records by batch.
- `payload_json` includes full snapshot.

Evaluation Harness

Ground Truth Generation

```
dataset = generate_batch(
    n_settlements=60,
    edge_cases={
        "clean_match": 30,
        "missing_reference": 5,
        "duplicate_settlement": 2,
        "bank_mismatch": 5,
        "fee_mismatch": 5,
        "tax_inconsistency": 3,
        "refund_timing": 5,
        "unexplained": 5
    }
)
```

Metrics

Metric	Formula
Match rate	`(TP + TN) / total`
False accept rate	`FP / total`

Table 10 – continued

Metric	Formula
Safe escalation rate	`(REVIEW_REQUIRED + UNRESOLVED) / total`
AI invocation rate	`AI_investigated / total`
AI auto-approval rate	`0` (by design)
Processing time	`batch_time / total`

Honest Reporting Template

> "We generated 60 settlements with known ground truth. The system correctly handled 43 (71.7% match rate). It escalated 14 to human review. It falsely accepted 2 settlements as clean when they had exceptions (5.0% false accept rate). AI was invoked on 8 settlements (13.3%). All AI-investigated discrepancies were flagged for human review; zero were auto-approved. Batch processed in 48 seconds (0.8s per settlement)."

File Structure

```
ledgermind/
├── backend/
│   ├── main.py                 # FastAPI app, endpoints
│   ├── models.py               # All Pydantic models
│   ├── ingestion.py            # CSV load, validate, normalize, idempotency
│   ├── linking.py              # Entity linker
│   ├── engine.py               # Deterministic reconciliation + rules
│   ├── ai_investigator.py      # Individual settlement AI
│   ├── batch_analyzer.py       # Batch-level AI pattern analysis
│   ├── audit.py                # Append-only audit logger
│   └── generator.py            # Synthetic data generator with ground truth
├── frontend/
│   ├── index.html
│   ├── App.jsx                 # Minimal React
│   └── components/
│       ├── UploadPanel.jsx
│       ├── ResultsTable.jsx
│       ├── ReviewQueue.jsx
│       ├── AuditTrace.jsx
│       └── BatchPatterns.jsx
├── data/
│   ├── demo/
│   └── evaluation/
├── tests/
│   ├── test_models.py
│   ├── test_ingestion.py
│   ├── test_engine.py
│   ├── test_ai.py
│   ├── test_batch_analysis.py
│   └── test_e2e.py
├── requirements.txt
```

```
└── README.md
```

Flat backend. No sub-packages.

Build Order

Phase	Component	Days
1	Data Models	0.5
2	Ingestion + Validation + Idempotency	1
3	Normalization + Entity Linking	0.5
4	Deterministic Reconciliation Engine	1
5	Synthetic Data Generator	0.5
6	Evaluation Harness	0.5
7	AI Investigator	1
8	Batch-Level AI Pattern Analysis	0.5
9	Audit Logger	0.5
10	FastAPI Endpoints	0.5
11	Minimal Frontend	1
12	End-to-End Tests + Demo Recording	2

Total: 10 days. Buffer: 2 days.

CRITICAL: Do NOT start with AI. Deterministic engine must be bulletproof first.

Test-First Checkpoints

Phase	Must Pass
1	All models serialize/deserialize. Invalid data raises ValidationError. Post-construction mutations rejected.
2	Valid CSV accepted. Invalid rejected with line numbers. Duplicate upload returns cached result. Encoding handled.
3	Dates ISO 8601. Amounts integer paise. Orphans, overclaims, mismatches detected. Bank linking by UTR + fallback works.
4	Hand-calculate 20 settlements. Engine matches exactly. Every deterministic exception triggers. Crash returns UNPROCESSED.

Table 12 – continued

Phase	Must Pass
5	Generator produces all 8 edge cases. Ground truth labels correct.
6	Evaluation harness scores batch correctly. False accept rate calculable.
7	LLM timeout → UNRESOLVED. Hallucinated evidence → UNRESOLVED. Valid evidence → correct classification. AI cannot alter expected_amount.
8	Batch analyzer detects at least one pattern on synthetic data.
9	Every settlement has one audit record. upload_hash groups by batch.
10	API accepts multipart upload. Returns job ID. Status endpoint returns results.
11	Dashboard shows upload → results → queue → audit trace.
12	60-record batch in <60s. 1,775 unexplained demo produces UNRESOLVED + human escalation.

UI Implementation Guidelines

Dashboard Hero Metrics

```
┌──────────────────────────────────────────┐
│  60  Settlements Processed               │
│  34  Clean                               │
│  20  Exceptions                          │
│   6  Unresolved                          │
│   0  Auto-Approved by AI                 │
│   8  AI Investigations (13.3%)           │
├──────────────────────────────────────────┤
│  🛡️  AI investigates. Humans decide.     │
└──────────────────────────────────────────┘
```

5-Screen Storytelling Flow

1. **"What happened?"** — Hero metrics
2. **"Why?"** — Click settlement → reconciliation trace
3. **"How did AI reason?"** — Evidence packet + AI explanation
4. **"Can I trust it?"** — Audit trail + safety guarantees
5. **"What did we learn?"** — Batch patterns + evaluation metrics

Reconciliation Trace

```
SETL_042 — Reconciliation Trace
```

```
Payments              ₹15,000
- Refunds              ₹2,000
- Fees                   ₹300
- Tax                     ₹54
──────────────────────────────
Expected              ₹12,646    ← Calculated by: Deterministic Engine

Actual                ₹10,871    ← From: settlements.csv
──────────────────────────────
Difference             ₹1,775    ← Calculated by: Deterministic Engine
                          ↓
                 MATH_DISCREPANCY
                          ↓
                  AI INVESTIGATION
                          ↓
                   UNRESOLVED
```

AI Boundary Indicator

Clean Match:

> 🟢 **AI not required** — Deterministic rules completely explain this settlement.

Fee Mismatch:

> 🟢 **AI not required** — Deterministic rule identified the exact violation.

Unexplained Gap:

> 🟣 **AI required** — Deterministic engine detected discrepancy but could not explain cause.

Safety Guarantees Panel

LedgerMind Guarantees

- AI never calculates money
- AI never modifies financial records
- AI never auto-approves
- Every AI claim must cite evidence
- LLM failures → human review
- All decisions are audited

Terminology Rules

- **"Deterministic fee validation"** — never "AI-powered fee validation"
- **"AI-powered discrepancy investigation"** — for AI's actual role
- **"Deterministic reconciliation engine"** — for Python math
- **"AI investigator"** — for LLM component

Demo Script

5 minutes. No cherry-picking.

Time	Action	Judge Sees
0:00	Upload 4 CSVs	Drag-and-drop, instant processing
0:15	Dashboard	**60 processed, 34 clean, 20 exceptions, 6 unresolved, 0 auto-approved, 8 AI invocations (13.3%)**
0:45	Click CLEAN_MATCH	Reconciliation trace. "AI not required. Deterministic rules explain this."
1:30	Click FEE_MISMATCH	Trace: fee expected 200, actual 201. "AI not required. Deterministic rule identified violation."
2:00	Click REFUND_TIMING	Evidence packet visible. AI: "Refund 2 min before cutoff." Confidence 0.82. Status: REVIEW_REQUIRED.
2:45	Human clicks "Approve"	Status → EXPLAINED. Audit trail records human action.
3:15	Click UNRESOLVED (1,775)	AI: "INSUFFICIENT EVIDENCE. ESCALATED." Confidence 0.15.
3:45	Show Safety Guarantees	"AI never calculates, never approves, never invents evidence."
4:00	Show Batch Patterns	"3 settlements on Aug 20 show 1-paise fee mismatches. Suggest reviewing fee rounding rule."
4:30	Show Audit Trail	Full JSON snapshot, timestamp, upload_hash, human action
4:45	Show Evaluation	"Ground truth: 60 labeled. Match rate 71.7%. False accept 5.0%. 0 auto-approved."

Scope Lock

NOT building:

- Auth, PostgreSQL, webhooks, notifications, OCR, multi-currency
- Recharts (unless time after Phase 12)
- LLM fine-tuning, crypto audit hashes, auto-approval, retry logic
- Complex React state (useContext max)

- Microservices, Kafka, Docker, Kubernetes
- Fancy animations, elaborate React architecture, more AI agents

---

AI Safety Invariants

Invariant	Enforcement
AI never calculates money	No financial fields in AIResponse schema. Python pre-computes all amounts.
AI never modifies financial records	AIResponse has no `amount`, `fee`, `refund`, or `tax` fields. `extra="forbid"` prevents injection.
AI never auto-approves	`recommended_action` is hardcoded to `ESCALATE_TO_HUMAN`. BatchMetrics and EvaluationResult enforce `auto_approved_by_ai == 0`.
AI must cite evidence	`cited_evidence` field validated against `evidence_packet.evidence_ids`. Missing citation → confidence 0.0.
LLM failures → human review	All failures (timeout, API error, malformed JSON, hallucination) → `UNRESOLVED` + human queue.
All decisions audited	Append-only SQLite records with full payload snapshot.

Failure Handling

LLM Failures

Failure	Behavior	Logged As
Timeout (>10s)	UNRESOLVED, confidence 0.0	`llm_error: timeout`
API error (5xx/4xx)	UNRESOLVED, confidence 0.0	`llm_error: api_error`
Rate limit	UNRESOLVED, confidence 0.0	`llm_error: rate_limit`
Malformed JSON	UNRESOLVED, confidence 0.0	`llm_error: malformed_json`
Invalid classification	UNRESOLVED, confidence 0.0	`llm_error: invalid_classification`
Hallucinated evidence	UNRESOLVED, confidence 0.0	`llm_error: hallucinated_evidence`
Ungrounded numbers	UNRESOLVED, confidence 0.0	`llm_error: ungrounded_number`

No retries. Fail safe immediately.

Deterministic Engine Crash

```
try:
    result = run_engine(settlement)
```

```
except Exception:
    result = {"decision": "UNPROCESSED", "escalate_to_human": True}
```

Never crash mid-batch. Never leave settlement in undefined state.

Duplicate Upload

```
def compute_upload_hash(file_paths: list[str]) -> str:
    contents = []
    for path in sorted(file_paths):
        df = pd.read_csv(path)
        df = df.sort_values(by=df.columns[0])
        contents.append(df.to_csv(index=False))
    normalized = "".join(contents).encode("utf-8")
    return hashlib.sha256(normalized).hexdigest()
```

If hash exists in `upload_log` → return cached result. No reprocessing.

Architecture Freeze Summary

Formula: `expected = sum(payments) - sum(refunds) - sum(fees) - sum(tax)` where `tax = floor(fee * 0.18)` and `fee = floor(amount * rate) + fixed`.

Deterministic engine: All arithmetic, all validation, all linking, all exception classification except TIMING_MISMATCH, REFUND_TIMING, UNEXPLAINED. Bank match: UTR primary, amount+date fallback.

AI investigator: Classifies discrepancies using structured evidence. Never calculates. Never approves. Never invents. All outputs → human review queue. Confidence = queue priority only.

Batch analyzer: Detects cross-settlement patterns (fee rounding, bank delay, refund cluster, unexplained gap). Does not approve.

Human: Final authority. Approves AI explanations to reach EXPLAINED state. Handles all deterministic exceptions.

Decision states: CLEAN_MATCH, DETERMINISTIC_EXCEPTION, REVIEW_REQUIRED, UNRESOLVED, EXPLAINED, UNPROCESSED.

Database: SQLite. Tables: `audit_log`, `upload_log`.

Backend: Python, FastAPI, Pydantic, Pandas. Flat files.

Frontend: Minimal React. 5-screen storytelling flow.

Audit: Append-only. UUID + timestamp + upload_hash + payload_json.

LLM failures: All → UNRESOLVED. No retries.

Build order: Models → Ingestion → Linking → Engine → Generator → Evaluation → AI → Batch Analyzer → Audit → API → Frontend → E2E Tests.

Evaluation: Ground truth synthetic data. Metrics: match_rate, false_accept_rate, safe_escalation_rate, ai_invocation_rate, ai_auto_approval_rate, processing_time.

ARCHITECTURE IS FROZEN

Begin coding. Do not revisit any decision above without explicit confirmation that a critical bug has been discovered during implementation.